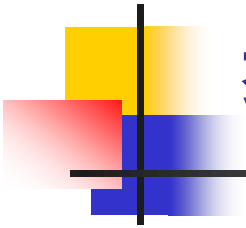


编译原理

中国海洋大学计算机系
葛 琳
gelin@ouc.edu.cn



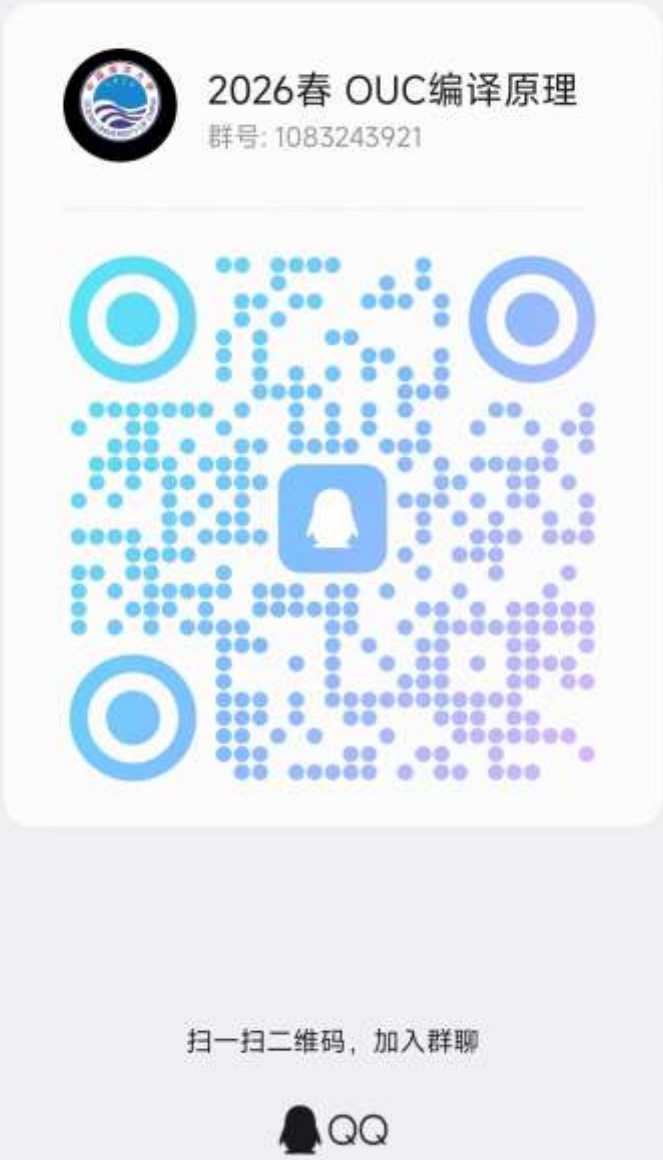
教师介绍

- 任课教师：葛琳
  - 办公地点：信息南楼B335
  - 答疑时间：周三上午9:30-11:30
  - Email：gelin@ouc.edu.cn
- 助教：
  - 赵朴一：信南B314，1619366358@qq.com

课程QQ群

- 加群密码：0403
- 进群请改昵称为：
  姓名 学号

如： 张三 19000001



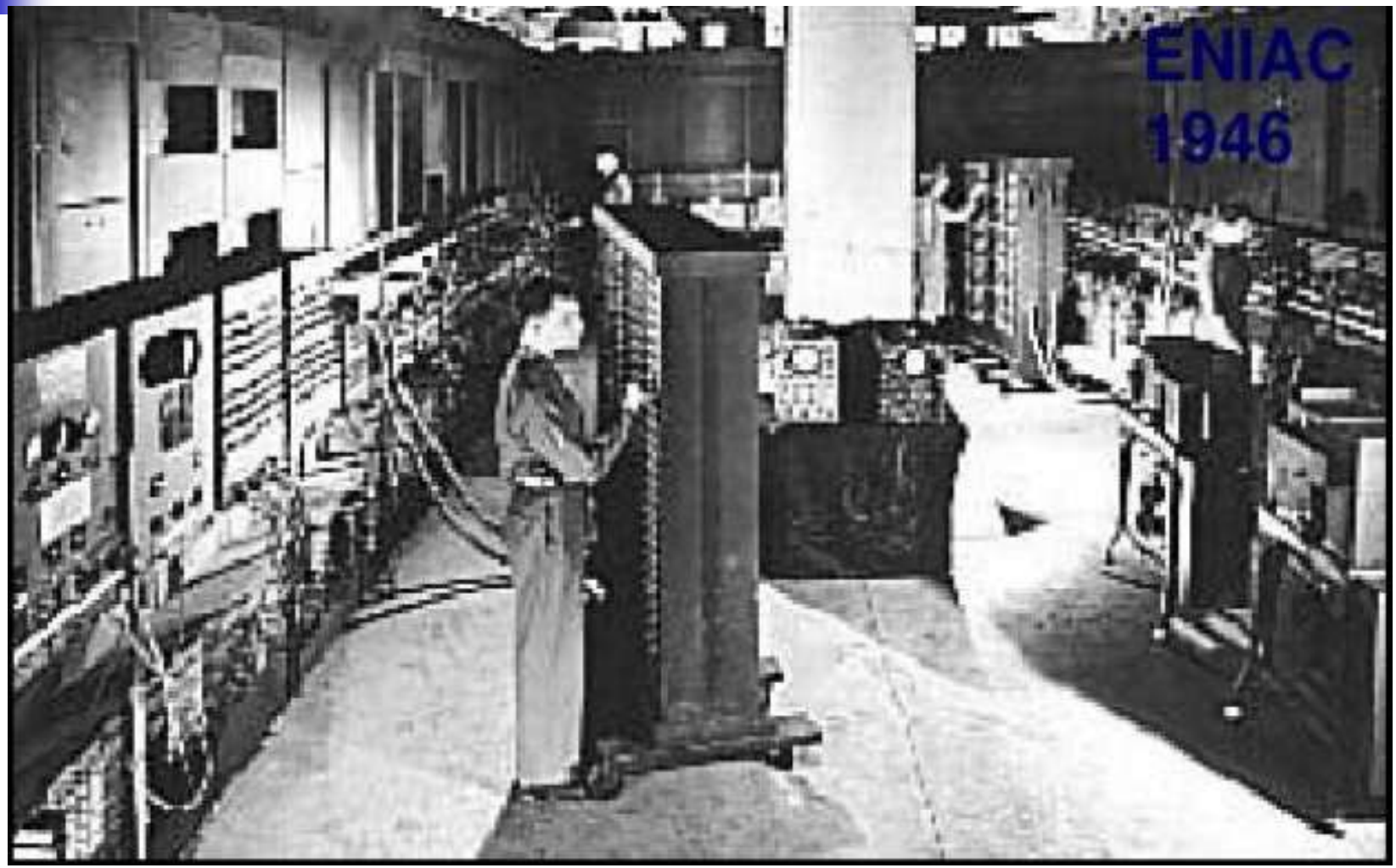
编译原理是计算机科学中历史最悠久的学科之一

- 计算机科学史上第一个完整功能编译器：Fortran I
  - Formula Translation
  - 1954-1957年，John Backus，for IBM 704

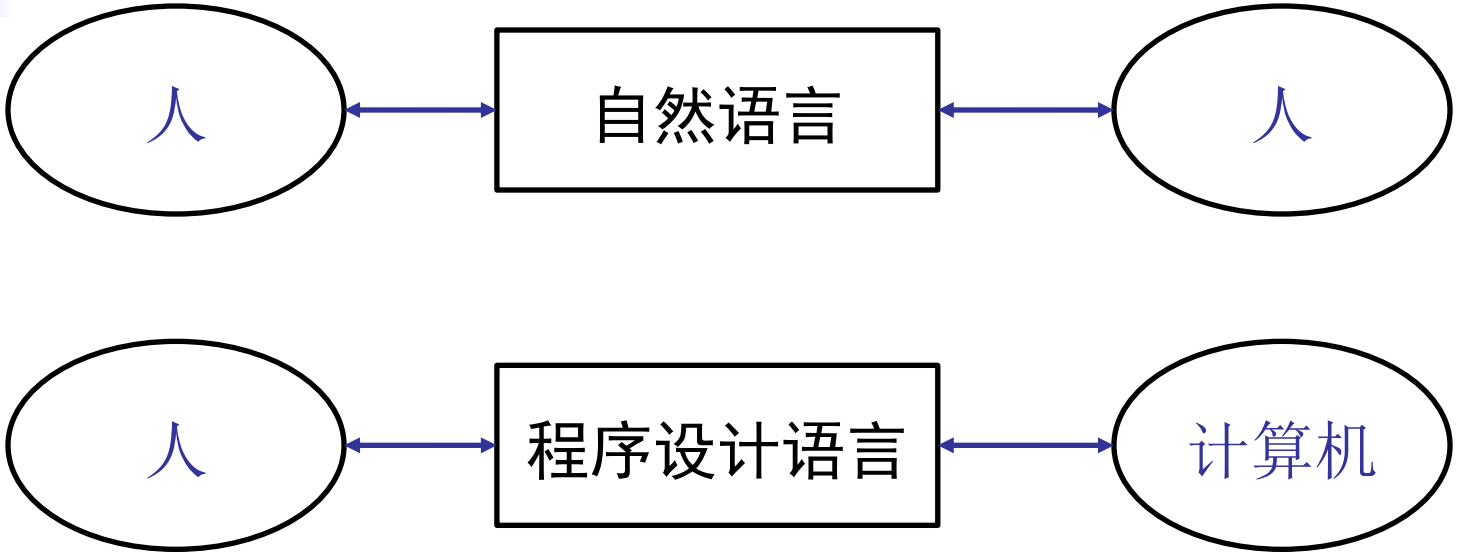




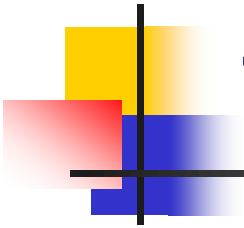
ENIAC-1946



程序设计语言

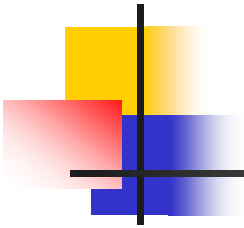


- 语言是人们沟通的媒介和手段
- 程序设计语言是人和计算机沟通的媒介和手段



计算机使用什么语言？

- 计算机使用机器语言（machine language）：
  - 机器语言程序本质上是一串0，1串，它可以被计算机理解并执行。
  - 人向计算机发送指令时，实际上需要发送一串由0、1组成的操作码和地址。
  - 人类理解机器语言比较困难，因而直接使用机器语言编程是困难而效率低下的。
  - 不同指令集架构的机器语言不一样（例如MIPS, X86，ARM……），和不同的机器“沟通”需要用不同的机器语言（即机器语言依赖于特定机器）。



汇编语言

- 为了增加人和计算机的沟通效率，人们用助记符来代替机器指令的操作码，用地址符号或标号代替指令或操作数的地址，这就形成了汇编语言（assembly language）。
  - MOV AL，num1 ；将 num1 的值传送到 AL 寄存器，对应机器码可能是A0 XX (XX是num1偏移)
  - ADD AL，num2 ；将 num2 的值加到 AL 寄存器中的值上，对应机器码可能是02 XX(XX是num2偏移)
- 汇编语言可以用汇编器通过简单变换变为计算机能够理解的机器语言。
- 汇编语言依然依赖于特定机器，人使用汇编语言效率依然不高。

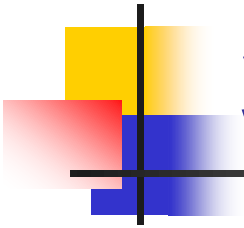
通常使用的程序设计语言

- 为了进一步增进沟通效率，人们希望程序设计语言进一步贴近人和人交流所使用的自然语言。
- 我们通常使用的程序设计语言（也叫编程语言，例如C、Java等）是高级语言（high-level language）：
  - 语义清晰，贴近自然语言（即人类使用的语言），易于理解，编程方便。
  - 语言不依赖于特定机器
  - 这样的程序无法被计算机直接理解和执行。

编译器的作用

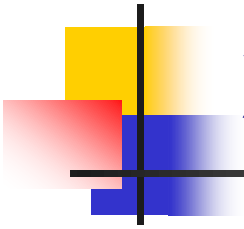


- 我们需要编译器来弥补人使用的高级语言和计算机使用的机器语言之间的鸿沟。



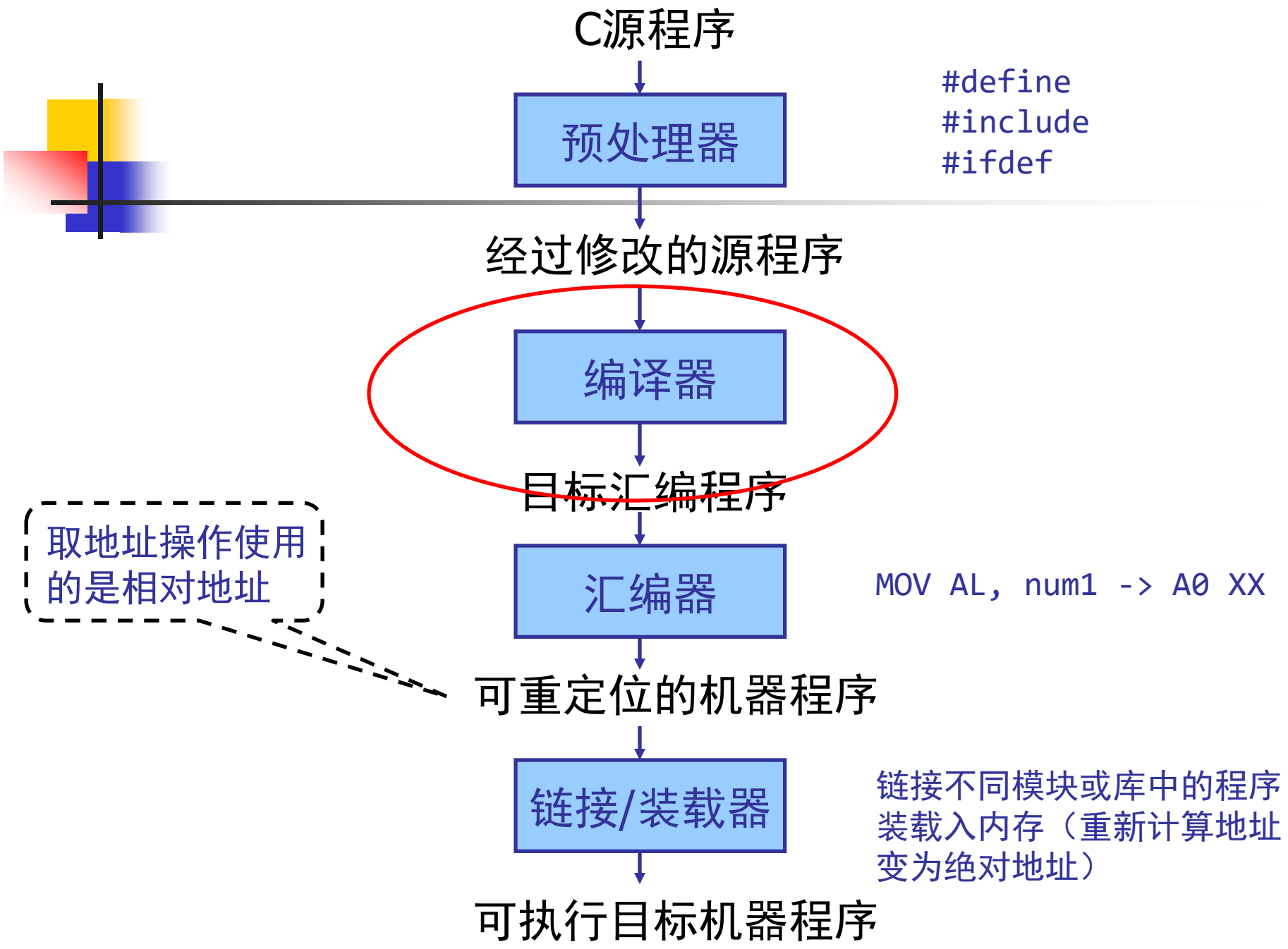
编译器的作用

- 我们用高级语言编写的程序只有被转换成机器语言程序后才能被计算机理解和执行。
- 这种转换主要是由编译器进行的。
- 在这一过程中，被转换程序称为源程序，其编程语言（通常是高级语言）称为源语言（source language），而转换后的程序称为目标程序，其语言称为目标语言（target language）。



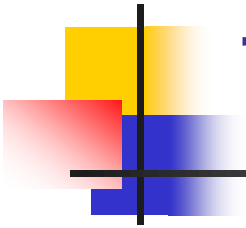
源程序如何成为可执行程序？

- 有了编译器的帮助后，高级语言编写的源程序，如何能够成为可以被机器执行的可执行程序？以你的理解说一说？（以大家熟悉的C语言为例）


C源程序
预处理器
#define
#include
#ifdef
经过修改的源程序
编译器
目标汇编程序
取地址操作使用的是相对地址
汇编器
MOV AL, num1 -> A0 XX
可重定位的机器程序
链接/装载器
链接不同模块或库中的程序装载入内存（重新计算地址变为绝对地址）
可执行目标机器程序

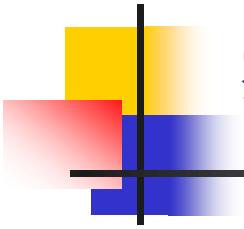
课程要求和教学目标

- 掌握编译器构造的一般原理和基本实现方法。
- 不偏向于某种源语言或目标机器，而是强调对编译原理和技术的宏观理解。



前驱课程：

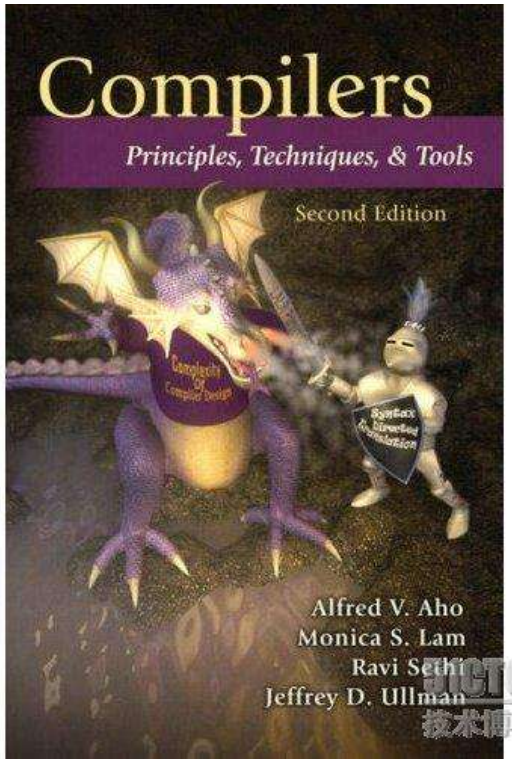
- 学习这门课程之前请确定你有一定计算机专业背景，并且已经学过下列课程：
  - 至少一门编程语言（比如C/C++/Java……）。
  - 数据结构、离散数学等，并有一定编程经验。
- 拥有下列知识将对你学习这门课程很有帮助：
  - 计算机系统原理、体系结构
  - 算法和软件工程
  - 更多编程语言知识（例如Pascal、Fortran……）

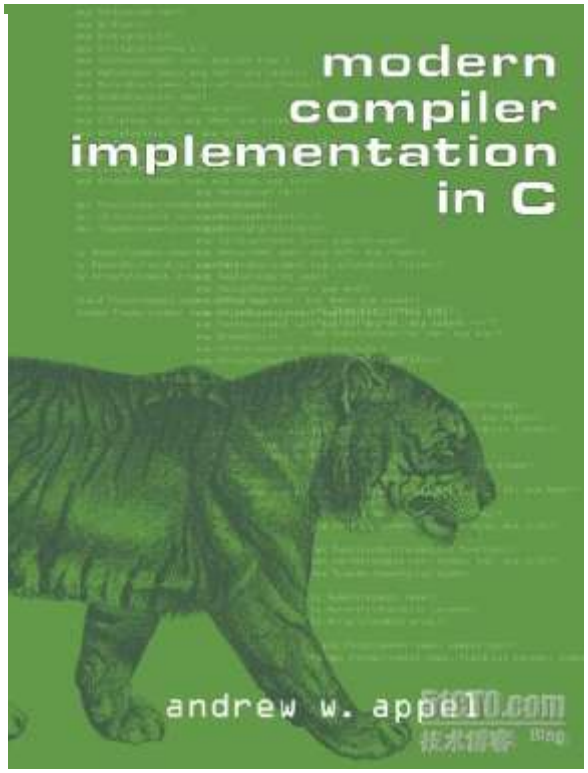


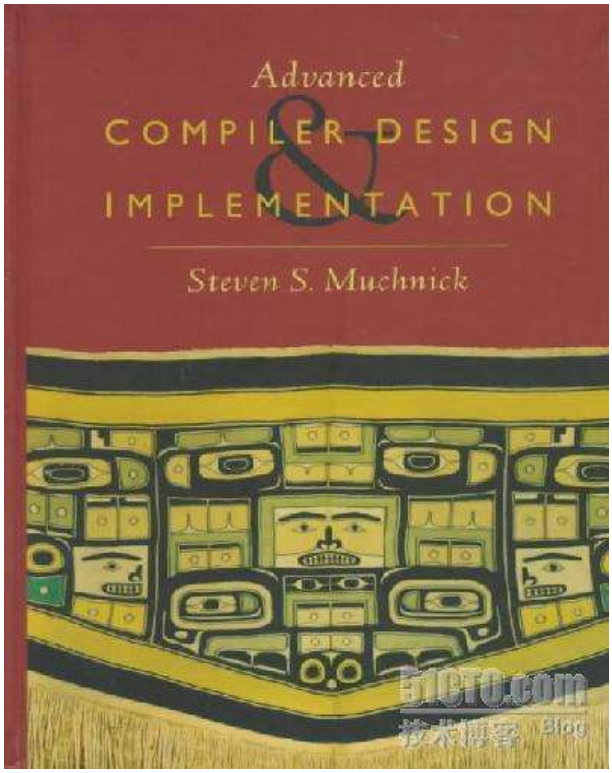
教材和参考书

- 教材：
  - 《编译原理》，陈意云、张昱，第3版，高等教育出版社，2014。
- 英文参考书：
  - Compilers: Principles, Techniques, and Tools. A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, 2nd edition, Addison-Wesley, 2007. （Dragon book）
  - Modern Compiler Implementation in C/Java/ML. Andrew W. Appel with Jens Palsberg. 第2版影印版，高教出版社。（Tiger book）
  - Advanced compiler design and implementation. Steven S. Muchnick. 影印版，机械工业出版社，2003。（Whale book）

教材和参考书（续）

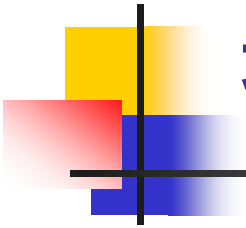

Compilers
Principles, Techniques, & Tools
Second Edition
Alfred V. Aho
Monica S. Lam
Ravi Sethi
Jeffrey D. Ullman


modern
compiler
implementation
in C
andrew w. appel


Advanced
COMPILER DESIGN
&
IMPLEMENTATION
Steven S. Muchnick

教材和参考书（续）

- 中文参考书：
  - 上述英文参考书的中文翻译版本。
  - 《编译原理》，蒋立源、康慕宁，第3版，西北工业大学出版社，2005。
  - 《程序设计语言编译原理》，陈火旺等，第3版，国防工业出版社，2000。
  - 《编译方法、技术与实践》，许畅、冯洋、郑艳伟、陈鄞 等编著，机械工业出版社，2024.



授课内容：

- 编译器概述（第一章）
- 词法分析（第二章）
- 上下文无关文法和语法分析（第三章）
- 语法制导翻译和类型检查（第四、五章部分内容）
- 运行时存储空间的组织和管理（第六章部分内容）
- 中间代码生成（第七章）
- 代码生成（第八章，根据时间调整）

课程要求：

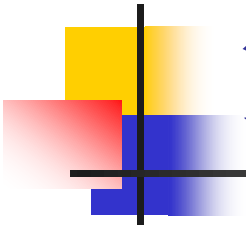
- 本课程共64学时，其中讲授48学时，实验16学时。
- 由于概念较多，因此要求大家：
  - 上课认真听讲，理解并牢记各种概念，带好教材和练习本（记录笔记或做课堂练习），及时完成课堂测试（课堂测试计入期末总分）。
  - 课前预习、课后复习并加深理解，多看书和课件。
  - 独立认真完成作业。
  - 课程实验：自己动手

考核要求

成绩综合评分体系	比例
1. 课堂测试（一般10%）、出勤、课后作业、课堂讨论、及平常表现	20%
2.实验程序及答辩表现、实验报告质量	20%
3.期末考试成绩	60%
总计	100%

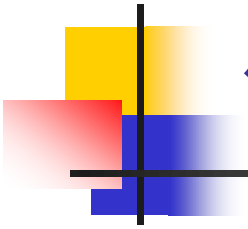
课程资源

- 雨课堂平台：
  - 课件、作业等会发布在雨课堂平台。课堂测试会在上课时通过雨课堂平台发布。
  - 作业和实验报告也在雨课堂平台提交。
- 头歌平台：
  - 编译原理-OUC-2026春-25352114
  - 邀请码：W7MBJ
  - 实验需要在头歌平台完成。



第一章 引论

- 什么是编译器？它和解释器的区别？
- 为什么要学习编译原理？
- 编译过程可以如何划分？各阶段的作用？
- 编译器是如何组织的？
- 编译器是如何产生的？

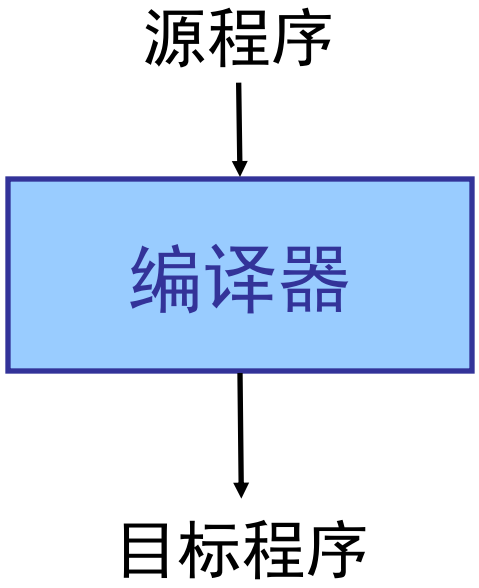


什么是编译器？

- 翻译器（translator）：
  - 能够将一种语言编写的程序翻译成语义等价的另外一种语言的程序的软件系统。
- 编译器（compiler）：
  - 翻译器的一种，通常其目标语言比源语言低级（低级语言：编程语言越接近机器语言，我们就说这种语言越低级。）

什么是编译器？

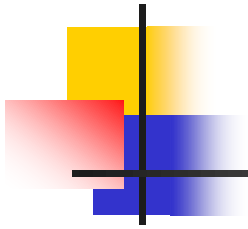
- 直观上理解的编译器就是这样一种程序：



编译器



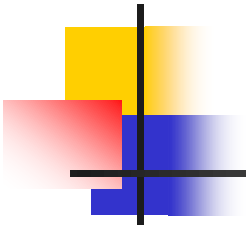
目标程序的运行



示例

```
int main()
{
  printf("hello, world\n");
  return 0;
}
```

```
    .text
str:
    .string "hello, world\n"
    .global main
main:
    pushl %ebp
    movl %esp, %ebp
    pushl $str
    call printf
    leave
    ret
```

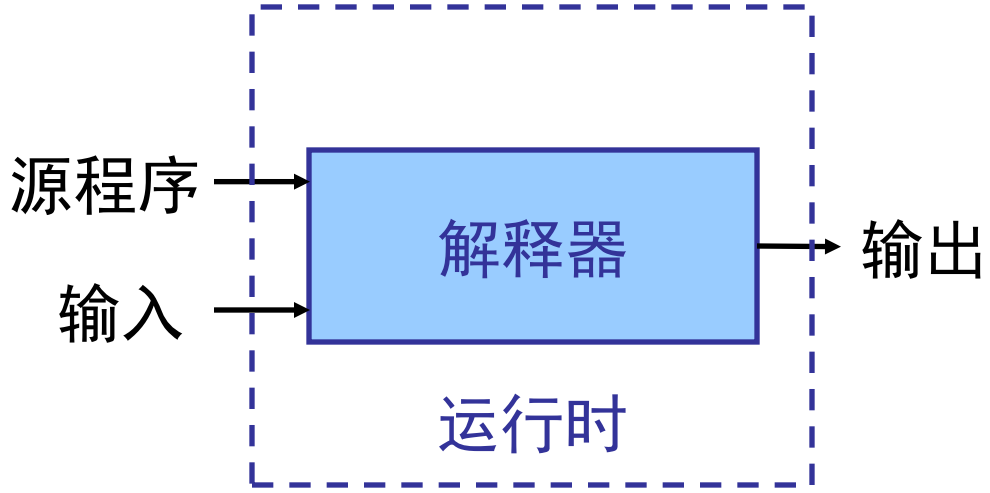


类比功能

- 我在小红书上教美国人做菜：
  - 我：写了一张中文菜谱
  - 翻译软件：把中文菜谱翻译成英文菜谱
  - 美国人：照着英文菜谱做菜。
- 类比编译器：
  - 中文菜谱：源程序
  - 英文菜谱：目标程序
  - 翻译软件：编译器
  - 美国人照着英文菜谱做菜：计算机运行目标程序
  - “放入大白菜……”：输入
  - 炒白菜出锅：输出

关于解释器

- 解释器（Interpreter）：不产生目标代码，而是解释执行源程序



类比功能

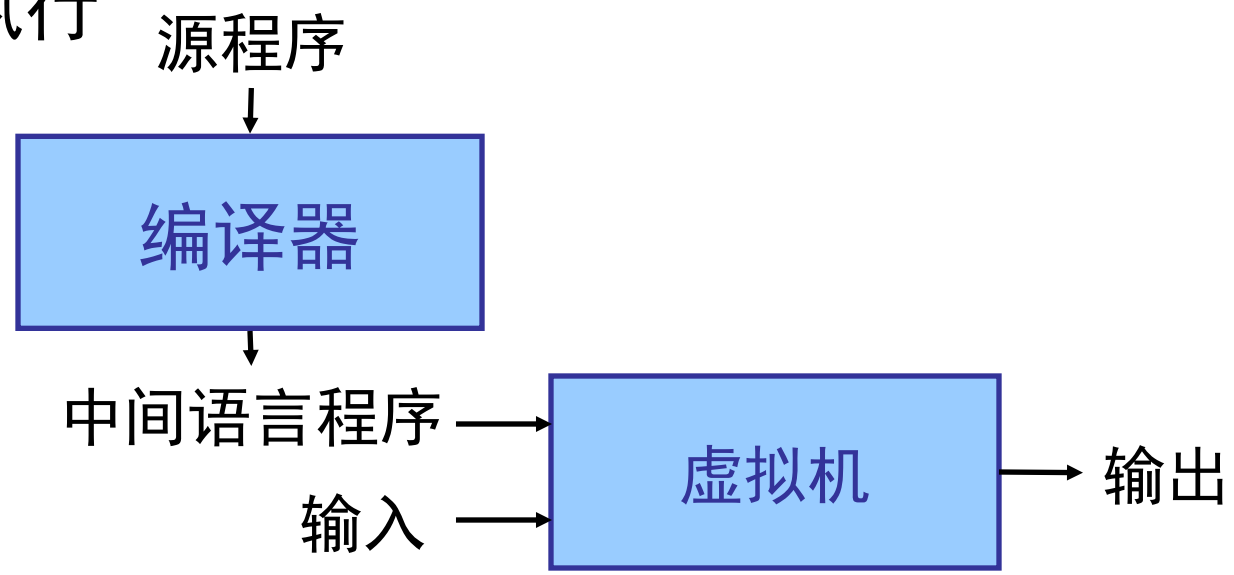
- 我在小红书上教美国人做菜：
  - 我：写了一张中文菜谱，直接发布在小红书上
  - 美国人：照着中文菜谱做菜。（需要一边用翻译软件翻译一边做菜）
- 类比解释器：
  - 中文菜谱：源程序
  - 美国人照着中文菜谱做菜：计算机对源程序进行解释执行。
  - 美国人的做菜过程：
    - “放油”翻译成英文“Add some oil”---- 操作放油……
    - “放入大白菜……”翻译成英文“put in the Chinese cabbage……”--- 操作放入大白菜……

编译器和解释器的优缺点？

- 编译器：（C/C++、Go、Fortran）
  - 执行效率高（运行时没有解释过程，目标代码可以进行全局优化）
  - 开发调试不够灵活（每次修改后需要重新编译整个程序）
- 解释器：（Python、JavaScript、Ruby）
  - 执行效率低（每次运行时都要重新解释，没有目标代码无法进行全局优化）
  - 开发调试更加灵活（随时修改随时看到修改效果）

编译器和解释器的综合应用

- Java语言将编译器和解释器综合使用：
  - 先用编译器把源程序翻译成一种叫做字节码（bytecode）的中间语言程序
  - 再用一个虚拟机（JVM）对字节码程序进行解释执行



为什么要学习编译原理？

- 新的程序设计语言的设计，编译器的构造
- 能够确切地知道你所编写的程序是如何从高级语言程序变成可执行程序的，从而对编程有更深入的理解；进而可以提高纠错能力和编写高质量代码的能力。这种能力对大型软件的构造和维护尤为重要。
- 编译原理集中体现了计算机科学的很多核心思想：算法、数据结构，软件工程等，是其他领域的重要研究基础。编译器设计中的一些原理和技术足以应用到你今后职业生涯中的很多领域。

为什么要学习编译原理？

- “落后就要挨打”是至理名言，任何时候，关键技术一定要掌握在中国人自己手中。
  - 2019年5月，美国商务部将华为列入 “实体清单”，限制美国企业向华为出售技术和产品，包括芯片等关键零部件。从此开启了一系列的针对中国的芯片技术的封锁政策。经过几年的封锁，生生将华为手机从高端手机市场赶了出去。
  - 2023 年 10 月 17 日，美国商务部工业和安全局（BIS）公布新的出口管制规则，禁止英伟达向中国出口 A800、H800、AD102 等高端芯片及 RTX4090 显卡。
  - 美国主流大语言模型（ChatGPT、Claude、Gemini等 ）均限制中国IP登录。

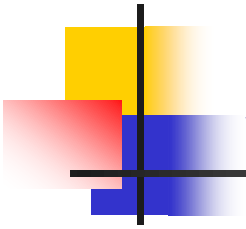
为什么要学习编译原理？

- 美国对中国的种种限制表明：美国正在试图锁死中国的科技发展之路。美国对中国芯片产业的打压和制裁让中国人彻底意识到，想要获得进一步的发展，科技创新和基础研究势在必行，关键技术一定要掌握在自己手中。我们不但要有自己的芯片还应该有自己的操作系统、编译器等系统软件。

为什么要学习编译原理？

- 值得庆幸的是，在中国科研人员、工程人员的不懈努力下，我们也取得了很多进步打破美国的技术封锁：
  - 2023年8月，华为发售Mate 60手机，采用自研国产芯片重回高端手机市场。
  - 2024年6月，华为发布全新编程语言 “仓颉”及其配套自研编译器（为配套鸿蒙原生应用开发)
  - 国产AI大模型技术蓬勃发展，2025年1月，国产大模型DeepSeek-R1开源。用更低的成本训练出不亚于ChatGPT的大模型。2026年2月，Seedance2.0视频生成模型横空出世，业界惊呼它对好莱坞成本与效率造成降维打击。
- 但是，由于我们基础薄弱，技术积累严重不足（我们在基础领域的经典课本都是国外教材或者改编自国外教材），需要我们在前辈的基础上继续努力奋斗！

为什么要学习编译原理？



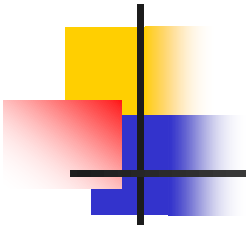
- AI时代来临了，我们还需要学习编译原理（、操作系统、体系结构）等计算机基础课程么？
  - 需要，而且很重要！AI 越强，懂底层、懂原理的人越不可替代！
    - AI会写代码片段，但是不会写完整系统（编译器、操作系统、高性能优化等），懂底层逻辑和懂架构的程序员不会被AI完全取代。
    - AI会有幻觉，你得知道它写的程序是对是错，错了你得知道错在哪里。
  - 为了不被AI替代，我们需要：懂底层+懂架构+懂AI

1.1 编译器概述

- 计算机科学史上第一个编译器：Fortran I
  - Formula Translation
  - 1954-1957年，John Backus，for IBM 704







1.1 编译器概述

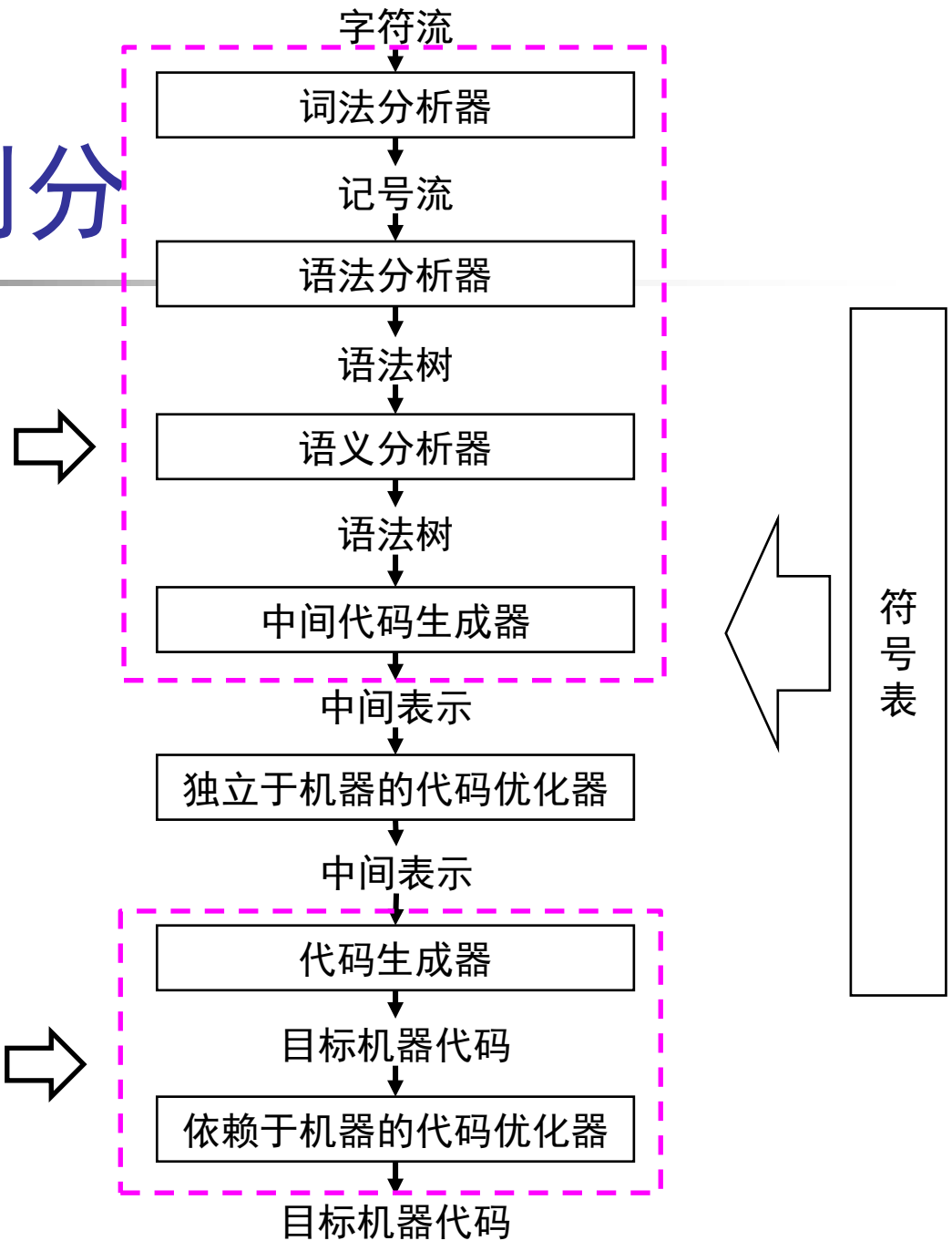
- FORTRAN I编译器的成功对计算机科学理论和实践都产生了重大影响：
  - 理论上：促进了算法、数据结构、形式语言与自动机理论等的发展
  - 实践上：现代编译器依然大体保持了FORTRAN I的架构

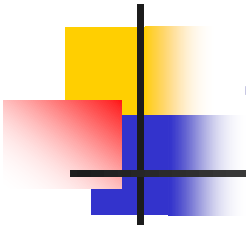
1.1 编译器概述

- 编译器的工作可以分成若干逻辑阶段，每个阶段把源程序从一种表示方式变换成另一种表示方式。
- 一个典型的阶段划分如下：

编译器的划分

- 前端（front-end）：分析部分，与源语言密切相关。
- 后端（back-end）：综合部分，与目标机器更相关





实例

- 下面我们用一个实例来说明编译器是如何将源程序变换为目标程序的，并借此例说明一下编译器各阶段都做了些什么。
- 下面这段程序只有一个赋值语句，变量的类型都为实型（浮点数）。

```
position = initial + rate * 60
```

1.1.1 词法分析（lexical analysis）

- 词法分析（又称线性分析或扫描scanning）：

  字符流 —词法规则→ 记号流（token）

- 记号：<记号名，属性值> 二元组

↓ 共用名称，表示类别　　↓ 词法单元的专属特征值

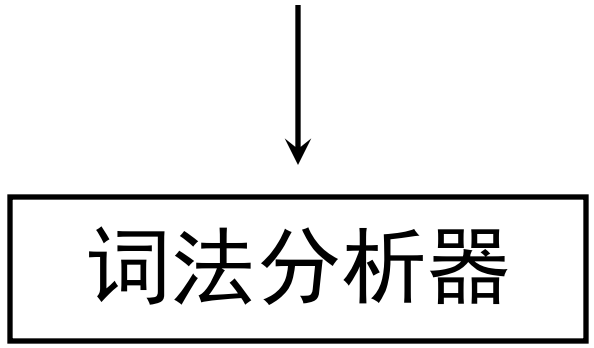
1.1.1 词法分析（续）

- 词法分析将在课本第二章详细介绍。
- 下面以实例来说明词法分析阶段是如何将字符流转换成记号流的：
  - 源程序字符流：

```
position = initial + rate * 60
```

1.1.1 词法分析（续）

```
position = initial + rate * 60
```



符 号 表

1	position	...
2	initial	...
3	rate	...

标识符的总称

只有一个实例，因而省略属性值

```
<id,1> <=> <id,2> <+> <id,3> <*> <60>
```

position在符号表中的条目

本应为<number, 条目>，简化

1.1.2 语法分析（parsing）

- 语法分析（syntax analysis），简称分析（parsing）。

  记号流 $\xrightarrow{\text{语法规则}}$ 树型抽象中间表示

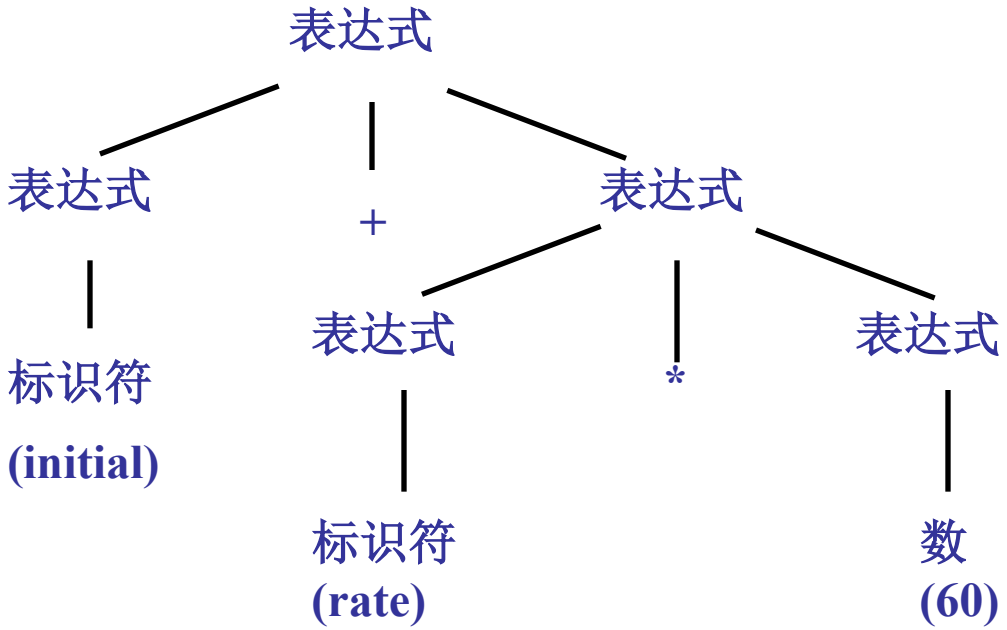
- 语法分析按编程语言的语法规则检查记号流是否满足这些规则，从而根据这些规则展示各种语言“构造”（construct，如函数、语句、表达式等）的层次性。

1.1.2 语法分析（续）

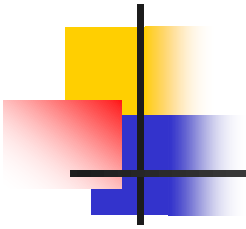
表达式的语法规则

- 任何一个标识符都是表达式
- 任何一个数都是表达式
- 如果e_1和e_2都是表达式，那么
  - $e_1 + e_2$
  - $e_1 * e_2$
  - (e_1)

也都是表达式



initial + rate * 60的分析树

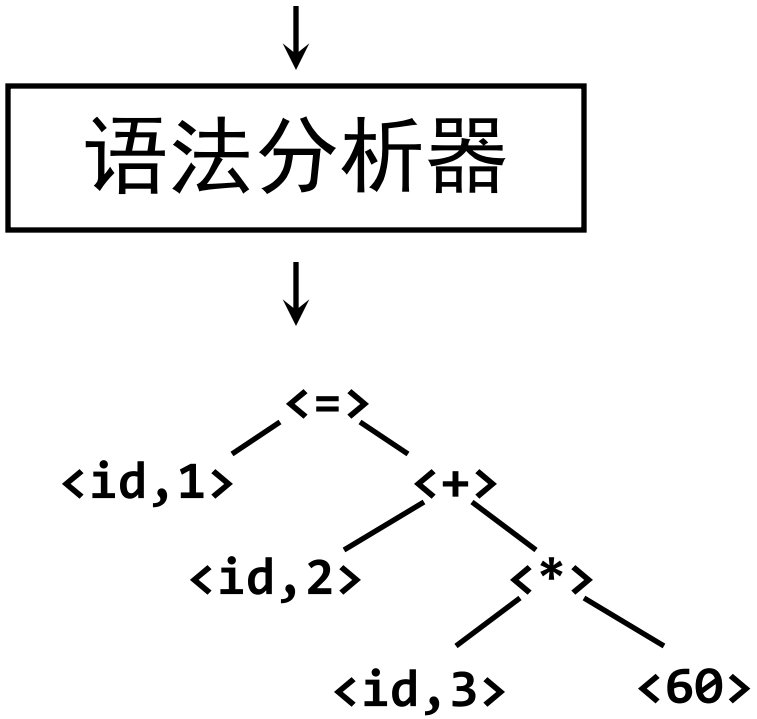


1.1.2 语法分析（续）

- 语法分析中使用的语法规则可以用上下文无关文法CFG（Context-Free Grammars）或Backus-Naur范式（BNF）来描述。
- 课本第三章介绍上下文无关文法和语法分析。
- 典型的中间表示是语法树

1.1.2 语法分析（续）

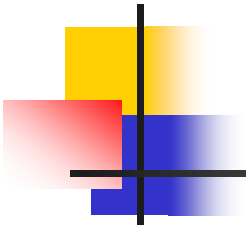
<id,1> <=> <id,2> <+> <id,3> <*> <60>



符 号 表

1	position	...
2	initial	...
3	rate	...

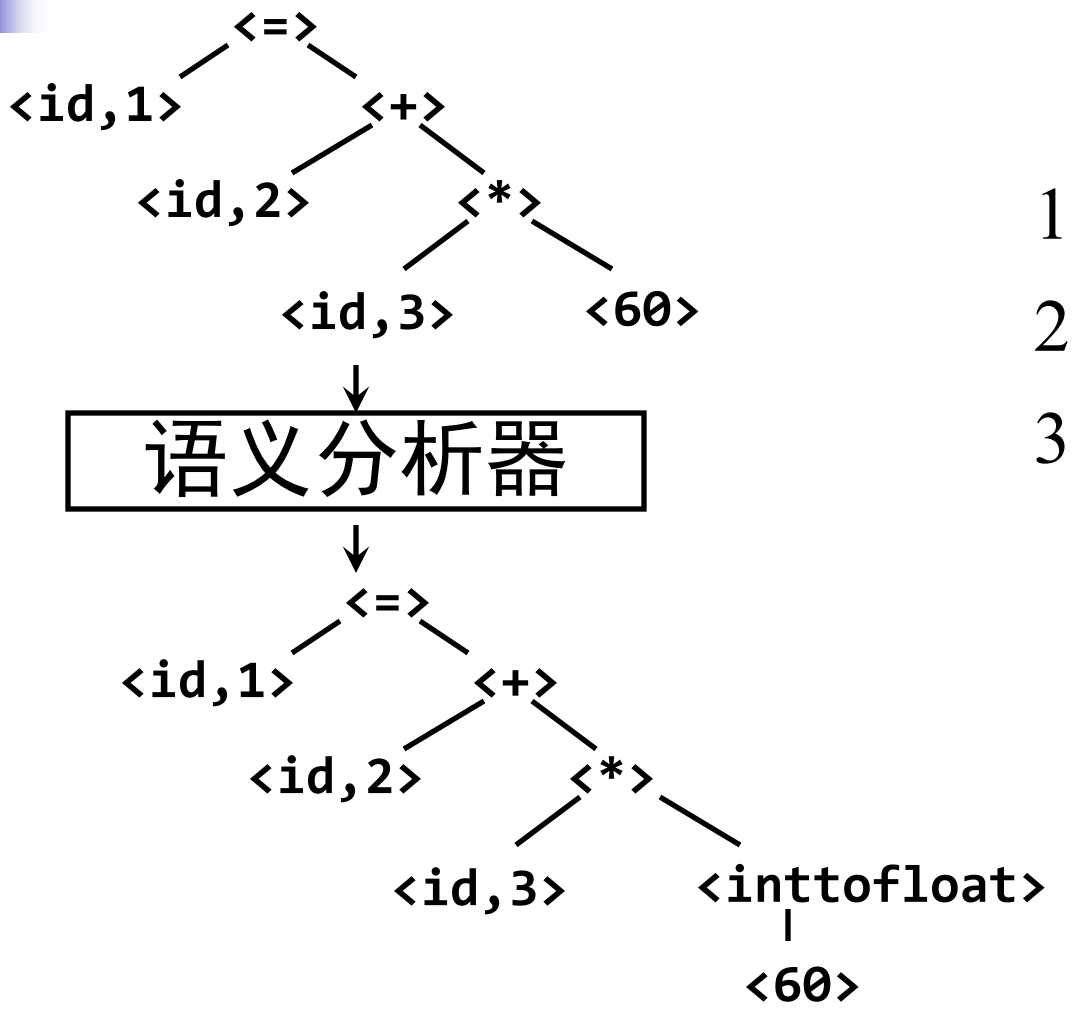
语法树



1.1.3 语义分析

- 语义分析：分析语法结构的含义和功能；检查程序语义一致性；为代码生成阶段收集信息。
- 语义分析的一个重要部分是类型检查。
  - 类型是否合法：例，数组下标为整数
  - 隐式类型转换（coercion）
- 目前较流行的是使用“语法制导翻译”的方法将语法分析和语义分析有机地组织起来。
- 课本第四章介绍语法制导翻译，第五章介绍类型检查。

1.1.3 语义分析（续）



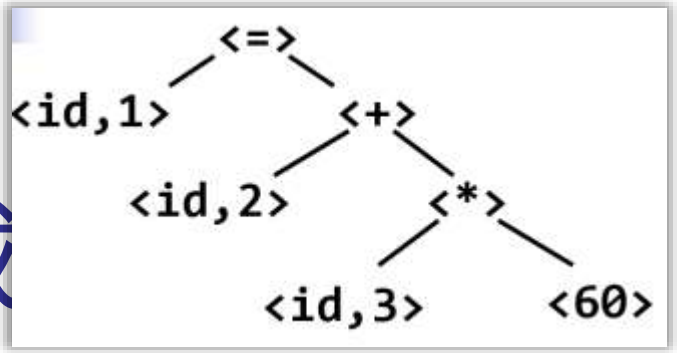
符 号 表

1	position	. . .
2	initial	. . .
3	rate	. . .

1.1.4 中间代码生成

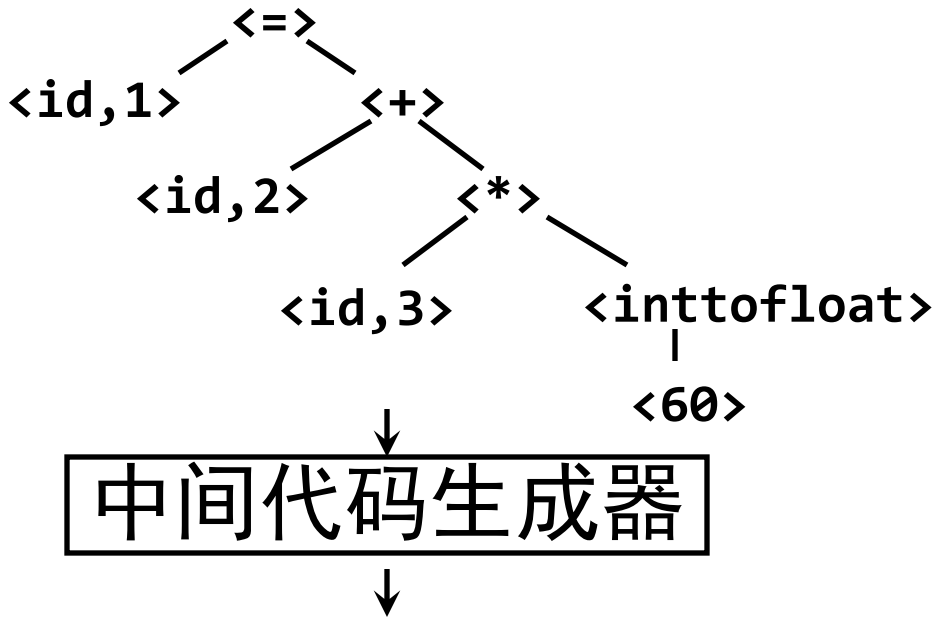
- 在对源程序进行过语法和语义分析之后，很多编译器会显式地生成一种比较低级的中间表示，即中间代码。
- 中间表示必须具备特点：
  - 易于产生。
  - 易于翻译成目标程序。

1.1.4 中间代码生成



- 中间表示有多种形式：逆波兰表示（后缀表示）、三地址码、树型结构等。
- position = initial + rate * 60的逆波兰表示为： position initial rate 60 * + =
- 一种很常用的中间表示是三地址码。

1.1.4 中间代码生成（续）




```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

符 号 表

1	position	...
2	initial	...
3	rate	...

特点：

- 必须先决定运算次序
- 临时变量
- 最多有三个运算对象

1.1.4 中间代码生成（续）

- 中间代码的产生与语义分析紧密相连，课本在第七章介绍中间代码生成。

1.1.5 代码优化

- 为了产生质量较高的目标代码，通常在中间代码生成和目标代码生成两个阶段之间，插入一个代码优化阶段。
- 质量较高的目标代码：通常指执行较快，但也可能指目标代码较短或执行时能耗较低。

1.1.5 代码优化（续）

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

↓

代码优化器

↓

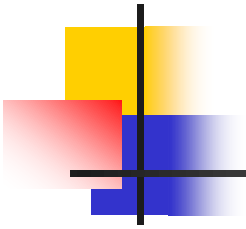
```
t1 = id3 * 60.0
id1 = id2 + t1
```

符 号 表

1	position	...
2	initial	...
3	rate	...

1.1.5 代码优化（续）

- 能够完成大多数优化的编译器叫做“优化编译器”。（编译时空复杂度增加，可靠性下降）
- 简单优化也可使目标程序运行时间大大缩短，而编译速度并未降低太多。
- 优化在课本第九、十章介绍，不作为本课程授课内容，感兴趣的同学请自行学习。



1.1.6 目标代码生成

- 代码生成阶段：由中间代码生成目标代码（机器指令或汇编码）
- 目标代码生成阶段所做的工作：
  - 为变量选择存储单元
  - 把中间代码翻译成等价的机器指令序列。
- 此阶段的关键问题是：寄存器分配
- 代码生成主要在课本第八章介绍，不作为本课程主要授课内容。

1.1.6 目标代码生成（续）

```
t1 = id3 * 60.0
id1 = id2 + t1
```

↓

代码生成器

↓

```
MOVF id3,  R2
MULF #60.0, R2
MOVF id2, R1
ADDF R2, R1
MOVF R1, id1
```

符 号 表

1	position	. . .
2	initial	. . .
3	rate	. . .

1.1.7 符号表管理

- 编译器的一项重要工作是记录并收集标识符及其各种属性，以提供标识符存储分配、类型、作用域等信息。
- 符号表为每个标识符保存一个记录数据结构，其中的域是标识符的各个属性。
- 该数据结构应允许编译器迅速找到一个名字的记录并读取和存储数据。

1.1.7 符号表管理（续）

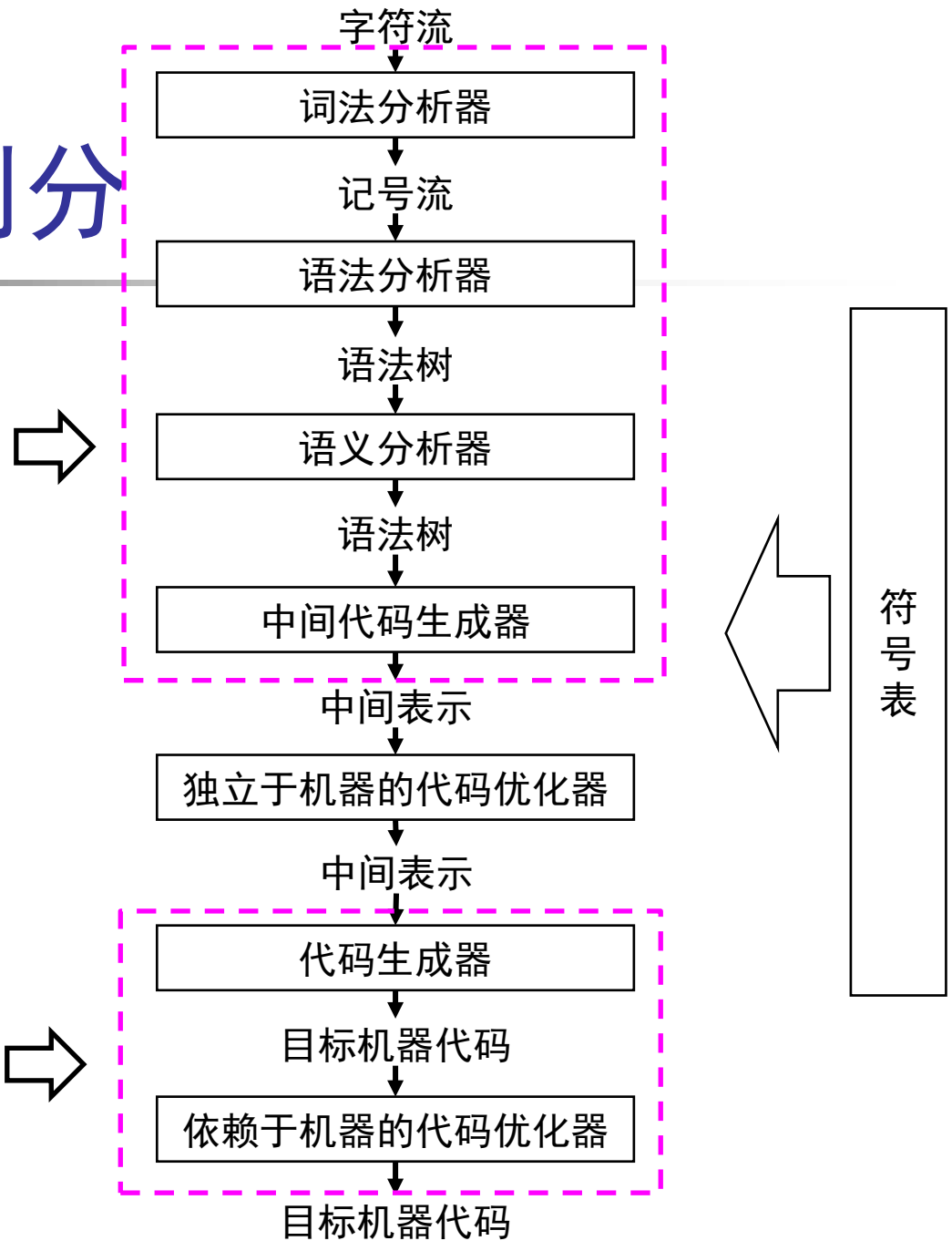
符 号 表

1	position	. . .
2	initial	. . .
3	rate	. . .

- 符号表将在课本第七章讨论。

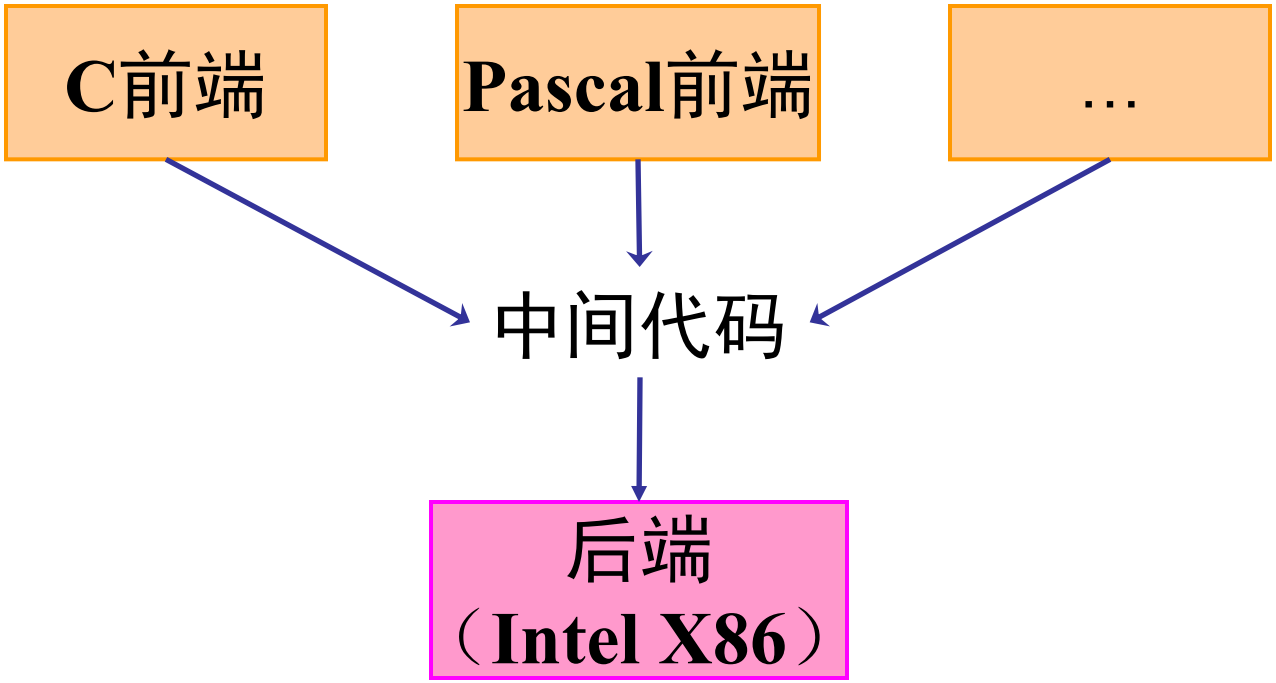
编译器的划分

- 前端（front-end）：分析部分，与源语言密切相关。
- 后端（back-end）：综合部分，与目标机器更相关



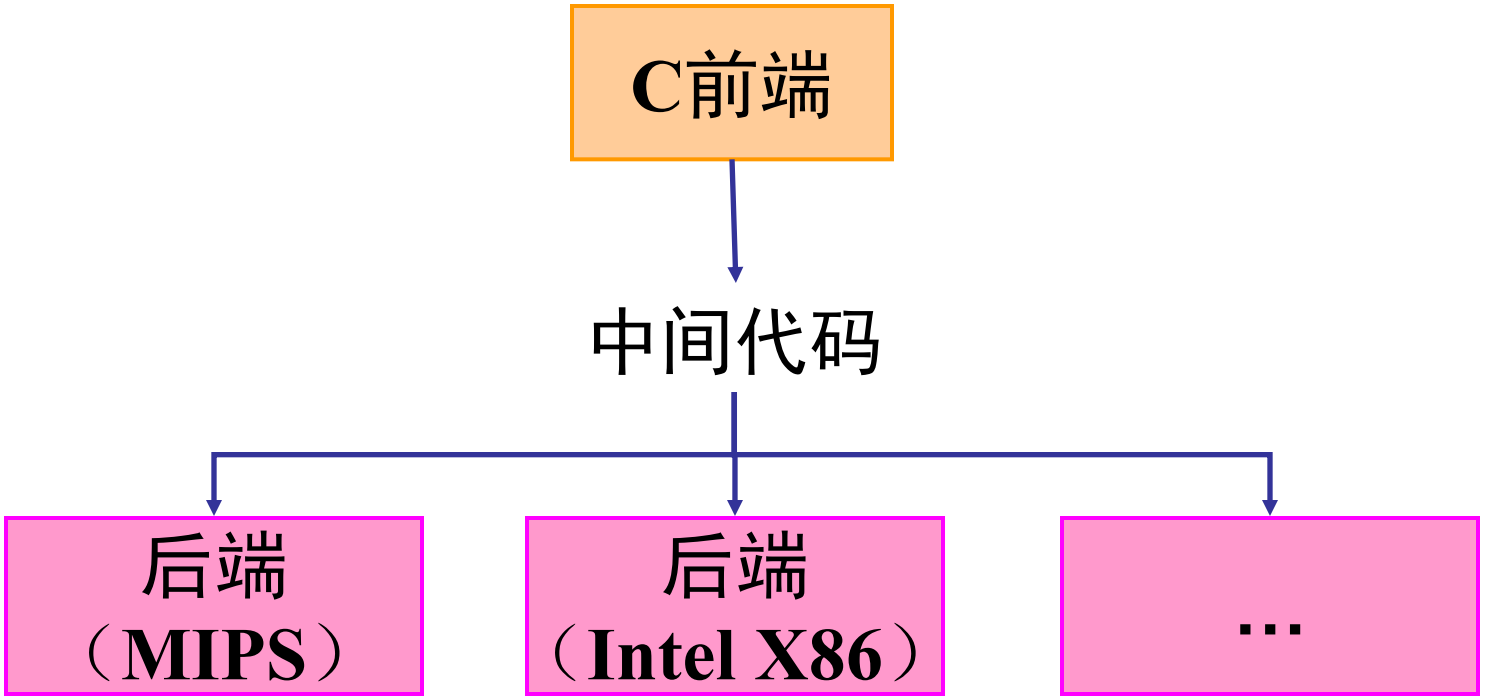
1.1.8 阶段的分组（续）

- 编译器划分前端后端有利于移植及简化编译器的设计

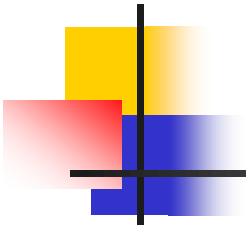


1.1.8 阶段的分组（续）

- 编译器划分前端后端有利于移植及简化编译器的设计



1.1.8 阶段的分组（续）



- 在编译器实现时，几个阶段的活动可能被组合起来形成一“遍”（pass）
- 每一“遍”都会读一个输入文件（例如源程序或源程序的中间表示），并写一个输出文件。
  - 例：前端的四个阶段：词法分析、语法分析、语义分析、中间代码生成可以组合形成一“遍”。
  - 例：代码优化可以进行很多“遍”。
- 现代高级语言大都不能通过一遍扫描就完成从源程序到目标代码的转换，因此一般都需要多遍扫描。

填空题 2分

设置

编译器的工作可以分成以下7个阶段：

A. 依赖于机器的代码优化器

B. 代码生成器

C. 独立于机器的代码优化器

D. 词法分析器

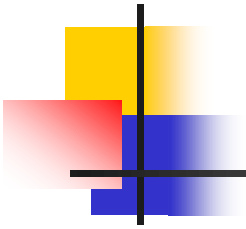
E. 语义分析器

F. 语法分析器

G. 中间代码生成器

请正确排列他们的顺序：(注意作答时只将字母按顺序排列，不要加空格和任何标点符号。正确填空方式类似：ABCDEFG)： [填空1]

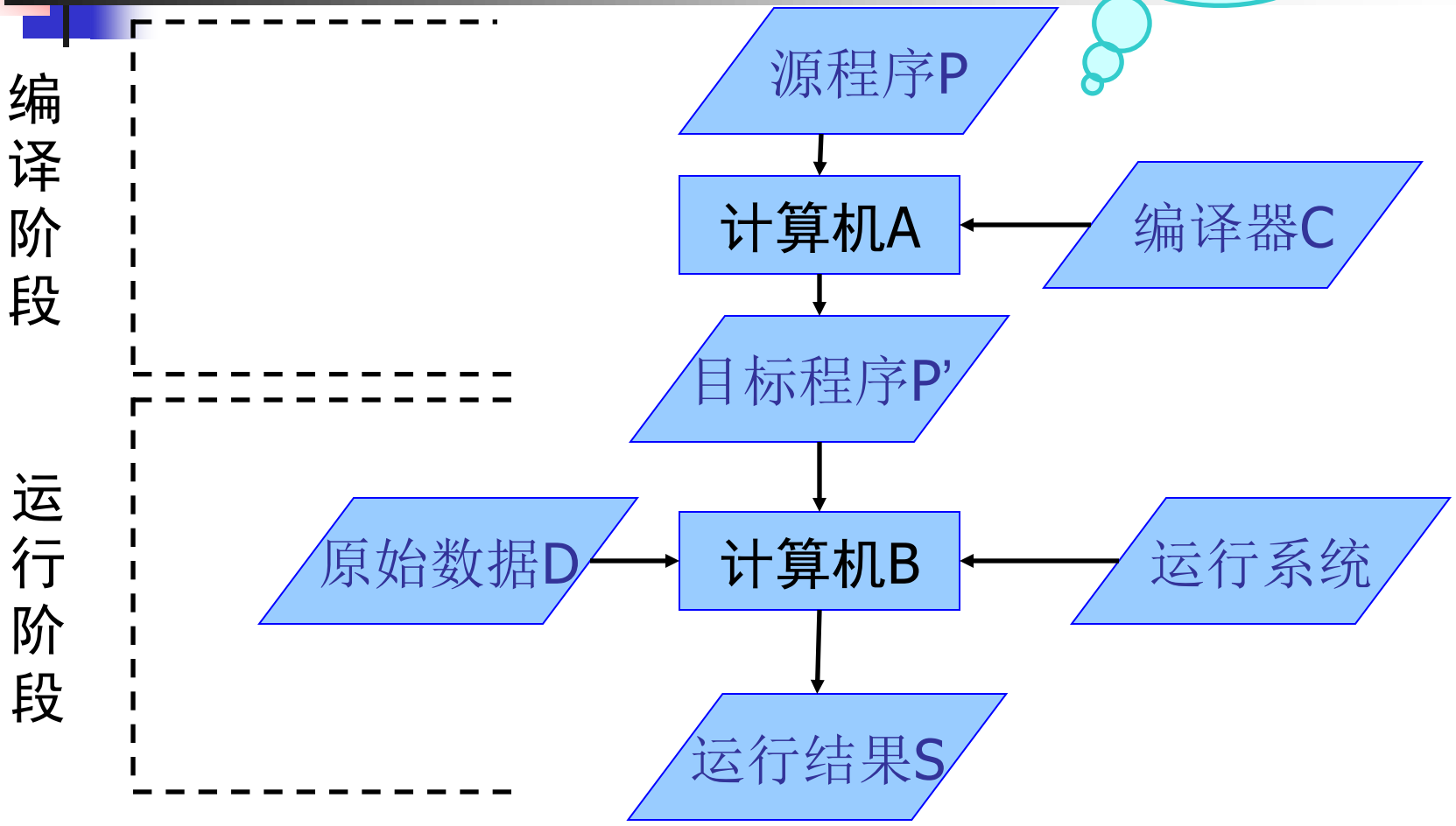
作答



1.2 编译器技术的应用

- 请自行看课本了解。

关于交叉编译


A和B是不同类型的计算机（指令集不同）
编译阶段
运行阶段
源程序P
计算机A
编译器C
目标程序P’
原始数据D
计算机B
运行系统
运行结果S

1.3 编译器的生成

- X86机器上原有一个C编译器(C→X86)，如何得到一个X86机器上的C++编译器(C++→X86)？
  - 写一个(C++→X86)编译器的C源程序，在X86机器上用(C→X86)编译器编译即可。

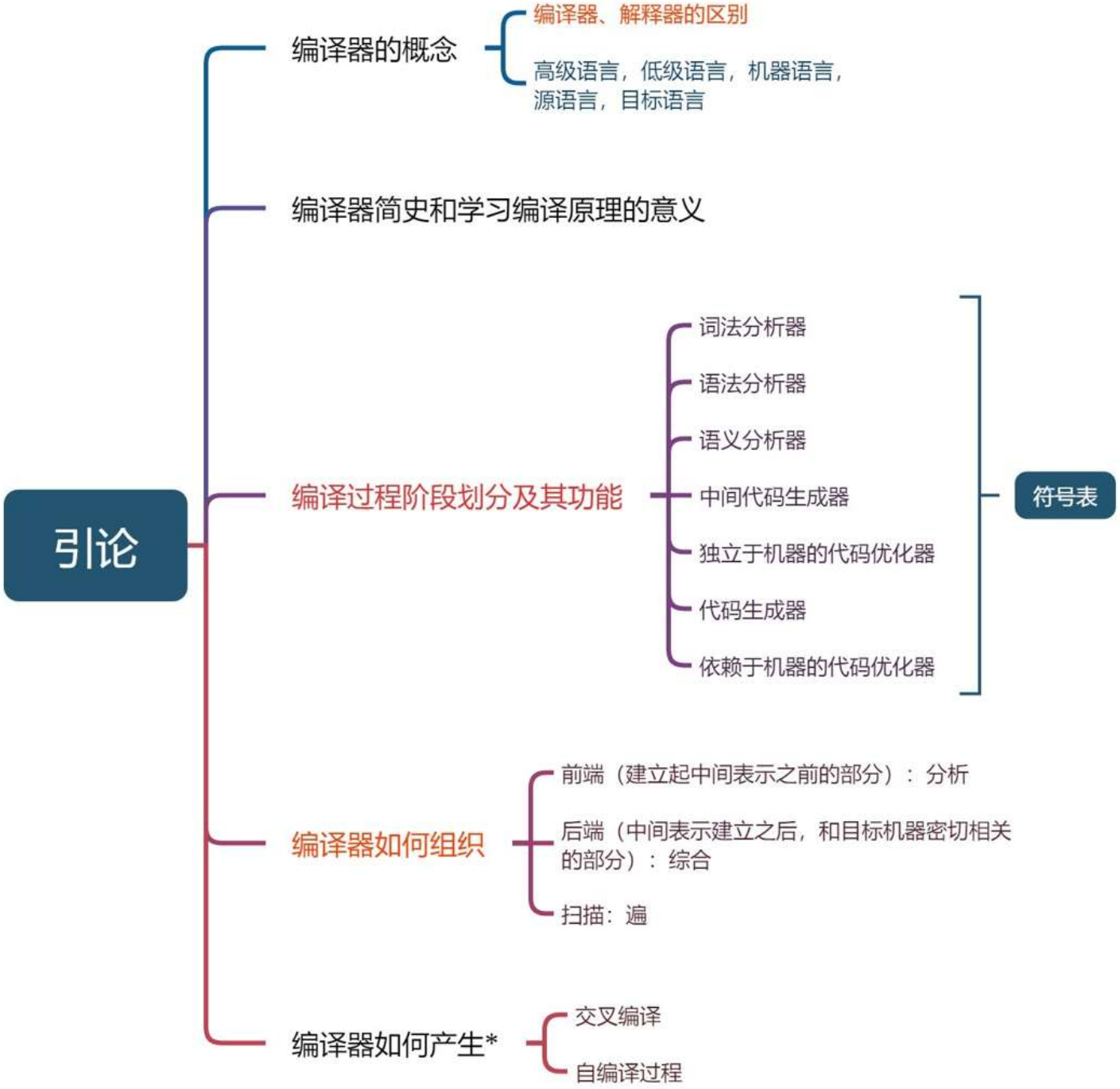
1.3 编译器的生成（续）

- X86机器上原有一个C编译器(C→X86)，如何得到一个MIPS机器上的C编译器(C→MIPS)？
  - 先写一个(C→MIPS)的C源程序
  - 在X86机器上用(C→X86)编译器编译该源程序得到X86机器上的(C→MIPS)编译器。
  - 在X86机器上用刚生成的(C→MIPS)编译器再编译C源程序一次，即得到MIPS机器上的(C→MIPS)编译器。

1.3 编译器的生成（续）

- X86机器上没有任何编译器，如何得到一个C编译器？（自编译过程）
  - 先对C语言的核心部分构造一个很小的编译器（用低级语言构造）
  - 用生成的这个编译器作为工具，继续构造一个稍大一些的编译器。
  - 如此扩展下去，直到得到一个完整的C编译器。

本章小结


引论
编译器的概念
编译器、解释器的区别
高级语言，低级语言，机器语言，源语言，目标语言
编译器简史和学习编译原理的意义
编译过程阶段划分及其功能
词法分析器
语法分析器
语义分析器
中间代码生成器
独立于机器的代码优化器
代码生成器
依赖于机器的代码优化器
符号表
编译器如何组织
前端（建立起中间表示之前的部分）：分析
后端（中间表示建立之后，和目标机器密切相关的部分）：综合
扫描：遍
编译器如何产生*
交叉编译
自编译过程

作业：（下周二之前完成）

- 在雨课堂平台上自由分组，每5-6人一组，以后将以该组为单位完成预习、讨论及部分作业。
- 以组为单位完成一篇博客发布在雨课堂讨论区，简单明了，不要长篇大论(2000字内)。可以从如下几个题目中选取一个：
  - 编译器技术可以应用在哪些领域？查找资料并组内讨论，然后发布博客。
  - 选择一种主流的（或你感兴趣的）程序设计语言（如Python），查找资料了解其发展历史及特点等，组内讨论后发布博客。
  - 选择一种主流的（或你感兴趣的）编译器（如GCC），了解其发展历史及特点等，组内讨论后发布博客。

博客命名为：“第N组1-1：XXX”，其中N为组号，XXX为你们组为博客起的题目（1-1表示第一周第一次课）。例如：第1组 1-1：漫谈GCC的前世今生。

和自然语言贴近的C语言程序

```c
#include <stdio.h>

int main() {
        int num;
        printf("请输入一个整数: ");
        scanf("%d", &num);
       if (num > 0) {
              printf("%d 是正数。\n", num); }
        else {
              printf("%d 不是正数。\n", num);
        }
        return 0;
}
```

返回